System Verification and Validation Plan for DAIIM

Team 6,
Robert Lam
Nathan Hum
Aidan Lao
Keira Laskoski

November 5, 2026

Revision History

Date	Version	Notes
Oct 26	1.0	Robert: Added system tests for authentication and task creation
Oct 27	1.1	Aidan: Added system tests for reputation and rewards, penalties, transparency, data handling, and safety; added non-functional tests, usability survey, test symbolic parameters, and reflection
Oct 28	1.2	Nathan: Added system tests for provider registraton and staking as well as inference requests and section 3, general information
Nov 1	1.3	Robert: Fixed formatting and typos. Added acronyms table. Added intro blurb.
Nov 1	1.4	Nathan: Edited some system tests for provider registration and staking.
Nov 1	1.4	Keira: Added the acceptance, prediction and aggregation system tests.
Nov 2	1.5	Keira: Clarify system tests and remove reference to specific blockchain networks.
Nov 2	2.0	Robert: Finalize formatting.

Table 1: Revision History

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 Introduction **1**

3 General Information **1**
- 3.1 Summary . . . 1
- 3.2 Objectives . . . 1
- 3.3 Custom Documents . . . 2
- 3.4 Relevant Documentation . . . 3

4 Plan **3**
- 4.1 Verification and Validation Team . . . 3
- 4.2 SRS Verification . . . 3
- 4.3 Design Verification . . . 4
- 4.4 Verification and Validation Plan Verification . . . 5
- 4.5 Implementation Verification . . . 5
- 4.6 Automated Testing and Verification Tools . . . 6
- 4.7 Software Validation . . . 6

5 System Tests **7**
- 5.1 Tests for Functional Requirements . . . 7
  - 5.1.1 Accounts and Authentication . . . 7
  - 5.1.2 Inference Task Creation . . . 11
  - 5.1.3 Provider Registration and Staking . . . 14
  - 5.1.4 Inference Requests . . . 18
  - 5.1.5 Provider Acceptance and Predictions . . . 22
  - 5.1.6 Aggregation and Results . . . 27
  - 5.1.7 Reputation, Rewards, and Penalties . . . 30
  - 5.1.8 Transparency and Data Handling . . . 35
  - 5.1.9 Safety . . . 37
- 5.2 Tests for Nonfunctional Requirements . . . 39
  - 5.2.1 Accuracy, Reliability, and Scalability . . . 39
  - 5.2.2 Usability . . . 40
  - 5.2.3 Security, Privacy, and Safety . . . 42
  - 5.2.4 Maintainability, Portability, and Installability . . . 43
- 5.3 Traceability Between Test Cases and Requirements . . . 43

6 Unit Test Description **45**

7 Appendix **46**
- 7.1 Symbolic Parameters . . . 46
- 7.2 Usability Survey Questions . . . 46

8 Appendix — Generative AI Use Disclosure **48**
8.1 AI Tools Used 48
8.2 How and Where Used 48
8.3 Verification of AI-Generated Content 48

9 Appendix — Reflection **49**

List of Tables

1 Revision History i
2 Symbols, Abbreviations, and Acronyms iv
3 Verification and Validation Team Roles 4
4 Traceability Between Test Cases and Requirements 44

1 Symbols, Abbreviations, and Acronyms

Table 2: Symbols, Abbreviations, and Acronyms

Term	Definition
CI/CD	Continuous Integration and Continuous Delivery, the automated process used to build, test, and prepare software changes for delivery.
FR	Functional Requirement, a requirement describing a behaviour the system must provide.
HA	Hazard Analysis, the analysis of potential system hazards, their causes, and their consequences.
HTTP	Hypertext Transfer Protocol, a protocol for transferring data between clients and servers.
NFR	Non-Functional Requirement, a requirement describing a quality, constraint, or operational property of the system.
NIST	National Institute of Standards and Technology, a United States government organization that publishes cybersecurity and technology guidance.
OWASP	Open Worldwide Application Security Project, an organization that publishes application-security guidance and testing resources.
PIPEDA	Personal Information Protection and Electronic Documents Act, a Canadian federal act governing the collection, use, and disclosure of personal information.
QA	Quality Assurance, the planned activities used to assess and improve software quality.
RPC	Remote Procedure Call, a mechanism used to invoke an operation on a remote service, such as a blockchain node.
SDK	Software Development Kit, a collection of tools and libraries used to develop software for a platform or service.
ST	System Test, an integrated test case that verifies system behaviour against one or more requirements.
TLS	Transport Layer Security, a cryptographic protocol for protecting data in transit and supporting authentication and integrity.

Term	Definition
Web3	Applications built on blockchain networks, where users interact with the system through their own blockchain accounts and authorize actions by signing transactions.

2 Introduction

This document presents the Verification and Validation (V&V) Plan for DAIIM. It describes the activities that will be used to increase confidence that the implemented system satisfies its functional and non-functional requirements. The plan identifies the documents, people, tools, environments, and testing methods that will support verification and validation throughout development.

The remainder of this document is organized as follows. The General Information section summarizes the system, defines the objectives of the V&V activities, and identifies the related project documents. The Plan section describes the V&V team and the planned approaches for verifying the SRS, the system design, this V&V plan, the implementation and the tools used in the validation process. The System Tests section specifies the tests of the integrated system, including tests for functional and non-functional requirements and the traceability between requirements and test cases. The Unit Test Description section outlines the scope and planned organization of unit-level testing and its relationship to the system modules. The appendices provide symbolic test parameters, usability survey questions, the Generative AI Use Disclosure, and a reflection on the V&V deliverable.

3 General Information

3.1 Summary

The software being tested is DAIIM (Decentralized Artificial Intelligence Inference Marketplace), a decentralized platform that allows users to submit artificial intelligence (AI) inference requests and receive aggregated predictions from multiple independent inference providers. The system enables task requesters to submit inference requests for supported machine learning tasks, while inference providers can register for tasks, contribute predictions, and receive compensation for their participation. The platform uses smart contracts deployed on a public blockchain to manage core marketplace operations, including provider registration, staking, request payments, result aggregation, reputation updates, and reward distribution. The system also includes a web-based user interface for interacting with the marketplace, an off-chain backend for coordinating inference requests and communicating with providers, and a database for storing relevant off-chain information. By combining on-chain and off-chain components, DAIIM aims to provide a transparent and decentralized marketplace that reduces reliance on centralized AI inference providers.

3.2 Objectives

The objective of this Verification and Validation (V&V) plan is to build confidence in the correctness, reliability, and usability of the DAIIM platform by verifying that its implemented components satisfy the requirements defined in the Software Requirements Specification (SRS).

Verification will focus on ensuring that the marketplace correctly handles the submission and validation of inference requests, provider registration and participation, prediction collection, result aggregation, and financial settlement. Particular attention will be given to the interactions between the on-chain smart contracts and off-chain components, ensuring that transactions and marketplace state changes are handled consistently and correctly.

Reliability will also be an important focus of the V&V activities. Testing will examine whether the system can produce aggregated results when multiple providers participate, including scenarios where some providers are unavailable or submit inaccurate predictions. This will help evaluate whether the aggregation and reputation mechanisms behave as intended under different provider conditions.

Usability validation will focus on whether users can interact with the marketplace without requiring extensive knowledge of blockchain technology or machine learning. This includes evaluating whether users can submit inference requests, authorize transactions, monitor request progress, and understand the resulting predictions and associated information.

The V&V plan will primarily be derived from the Software Requirements Specification (SRS), which defines the functional and non-functional requirements used to construct system-level test cases. The Hazard Analysis will also guide testing priorities by identifying potential failures involving inference reliability, transaction integrity, security, availability, and incentive mechanisms. System architecture and design documentation will further support test development by identifying component interactions, data flows, and interfaces that require verification.

Due to project time and resource constraints, certain verification activities will be outside the scope of this plan. The internal correctness of external dependencies, such as the blockchain network, wallet software, and third-party infrastructure, will not be independently verified. Instead, testing will focus on ensuring that DAIIM integrates with and uses these dependencies correctly. Similarly, the internal implementation and predictive capabilities of individual providers' machine learning models will not be independently validated. Testing will instead focus on the collection and aggregation of provider predictions and the resulting marketplace behaviour.

3.3 Custom Documents

The project includes two approved Custom Documents (CDs):

- **Cognitive Walkthrough:** A cognitive walkthrough will be conducted using a wireframe prototype of the planned application interface, from the perspective of inference requesters. This activity will help identify usability issues and areas of ambiguity in the user workflow. The findings will support validation of whether the interface is understandable and learnable, particularly for users with limited experience in blockchain technology or machine learning.

- **Business Plan:** A business plan will be developed following the completion of a successful proof of concept. It will describe the proposed business model and market

strategy for DAIIM, including the potential market need, cost breakdown, competition, tokenomics, and execution plan. Although this document does not directly contribute to software verification, it provides additional context regarding the intended operation and long-term objectives of the marketplace.

3.4 Relevant Documentation

The following documents provide essential background and guidance for the V&V activities:

1. **Software Requirements Specification (SRS):** The SRS defines the functional and non-functional requirements of DAIIM. It serves as the primary reference for developing system-level test cases and determining the expected behaviour of the implemented software.

2. **Hazard Analysis (HA):** The Hazard Analysis identifies potential system-level hazards, their causes, and their consequences. It helps prioritize verification activities toward areas where failures could have significant effects, including inference accuracy and reliability, transaction integrity, provider security, and financial settlement.

4 Plan

4.1 Verification and Validation Team

All members, aside from the supervisor, will share the responsibilities of designing the system, writing tests, and developing the system.

4.2 SRS Verification

The SRS document will be verified to ensure requirements are unambiguous, verifiable, and complete.

- **Structured Review with Supervisor:** A structured meeting with the supervisor will occur. The SRS document will be shared with the supervisor and requirements will be reviewed. The team will prepare questions in advance.

- **Ad Hoc Peer Review and Inspection:** Classmates and primary reviewers will be recruited to perform an ad hoc review of the SRS document. All criticisms will be documented as issues and addressed by a member of the team. A minimum of 5 issues created based on reviews from peers will be the objective.

- **Issue Tracker for Traceability:** As stated above, a GitHub issue tracker will be used to trace criticisms and suggestions for the amelioration of the SRS document. All issues will be reviewed, changes to the document will be implemented if necessary. All issues will be addressed with a comment and closed.

Team Member	Project Role	VnV Responsibilities
Robert Lam	Team Lead	Schedules reviews, demos and the usability sessions. Runs the structured reviews with Dr. Hassan. Decides when a deliverable's testing is done.
Nathan Hum	DevOps and Test Tooling Lead	Maintains GitHub Actions, the linters, coverage reports and branch protection. Maintains the local validator, the funded Devnet test wallets, and shared test fixtures.
Aidan Lao	QA and Inspection Lead	Keeps the review checklists and runs the checklist inspections. Assigns cross-reviews so nobody verifies their own work, for example the reference implementation and the security audit. Makes sure every pull request gets its reviewers and every issue gets closed.
Keira Laskoski	Traceability and Documentation Manager	Records the defects and decisions from reviews. Keeps the traceability tables up to date. Checks that FR numbers match between the SRS and VnV after every revision. Create and run the Cognitive Walkthrough.
Dr. Mohamed Hassan	Supervisor and Stakeholder Representative	Technical expert and advisor. Reviews team documents and offer technical insight and expertise. Make suggestions from the point of view of the stakeholder.

Table 3: Verification and Validation Team Roles

- **Checklist-Based Inspection:** The team will prepare a checklist for necessary characteristics for a high quality issue. All requirements will be reviewed by team members to ensure each possesses these characteristics.

- **Consistency and Traceability Check:** The team will check that the use cases agree with the functional requirements, and that every functional requirement appears in the traceability tables. For example, the use case for accepting an inference request must describe the same staking behaviour as the staking and penalty requirements. The team will also check that requirement numbers in the SRS match the numbers used in this plan after each revision of the SRS.

4.3 Design Verification

- **Peer Review and Inspection:** Classmates and primary reviewers will be recruited to perform an ad hoc review of the first edition of the design document after the course-imposed deadline. All criticisms will be documented as issues and addressed by a member of the team. A minimum of 5 issues created based on reviews from peers and primary reviewers will be the objective.

- **Team Member Review:** All team members will review the design document to

ensure the design meets the requirements, is feasible, and is properly decomposed.

- **Checklist-Based:** The team will check the design documents against a checklist. The checklist will confirm that each functional and non-functional requirement is assigned to at least one module, that each module's interface provides the access programs needed to satisfy its requirements, and that each on-chain instruction checks the signatures and balances required by the safety requirements (FR.SA.1 to FR.SA.10).

4.4 Verification and Validation Plan Verification

- **Structured Review with Supervisor:** A structured review of the VnV plan will be performed with the supervisor, once completed. Team members will describe key components of the plan and the test environments they rely on, such as Solana Devnet and the simulation harness, and ask the supervisor whether the tests check the behaviour that matters most to requesters and providers. Feedback will be recorded as GitHub issues.

- **Peer Review and Inspection:** Classmates and primary reviewers will be recruited to perform an ad hoc review of the VnV plan. All criticisms will be documented as issues and addressed by a member of the team.

- **Team Member Review:** Each team member will review the tests in a section they did not write, checking that the expected outputs follow from the requirements and that each test can be run as described. Tests that rely on Solana Devnet will be reviewed for timing assumptions, including the task acceptance period and protocol-wide commit and reveal periods.

- **Checklist-Based Inspection:** The team will check that every functional and non-functional requirement in the SRS is named in the test case derivation of at least one test, and record the result in the traceability tables. Requirements that are only partly tested will be listed so that the missing cases can be added.

4.5 Implementation Verification

- **Unit and System Tests:** All functional and non-functional requirements will be tested according to the described system tests (Section 5) and unit tests (Section 6) of this document.

- **Static Analysis and Linters:** The team will enforce code standards using linters and static analysers.

- **Code Review:** Every change will be merged into `main` through a pull request reviewed and approved by at least one other team member, implemented through CI/CD pipeline on GitHub. Reviewers will check the coding standards that linters cannot, such as documentation comments, and that the change includes tests.

4.6 Automated Testing and Verification Tools

The team will use the following tools. The coding standards and the tools that enforce them are set out in the Development Plan.

- **Anchor and Solana:** Used to test the on-chain programs. The tests set up accounts, run each instruction, and check account state before and after the transaction.
- **Solana Devnet:** Used for the system tests that need a real network, with test wallets funded with Devnet SOL.
- **Vitest:** Used for the unit tests of the React frontend and other TypeScript code.
- **Automated Browser Tests:** Used to run the main user flows in Chrome, Firefox, and Edge, signing transactions with a test wallet signer. Safari is tested by hand.
- **Linters and Formatters:** ESLint for TypeScript, and `cargo clippy` for Rust.
- **Continuous Integration (CI):** GitHub Actions are implemented to run automated tests, linters, and builds on every pull request. Branch protection on `main` prevents merging while any check fails.
- **Code Coverage:** Vitest's coverage report for TypeScript and `cargo llvm-cov` for Rust. The team aims for at least 80% line coverage of the main logic modules.

4.7 Software Validation

Validation checks that DAIIM meets the needs of its requesters and providers, not only that it matches the SRS. There is no external data set for validating DAIIM, so validation relies on the supervisor, on users, and on an independent reference implementation of the aggregation algorithm.

- **Supervisor Demonstration:** The team will demonstrate the system to Dr. Hassan shortly after the scheduled Rev 0 demo. The demonstration will follow a requester and a group of providers through a complete inference request on Solana Devnet, including at least one failure case, such as a request cancelled for too few providers. The supervisor's feedback will be recorded as GitHub issues, and any change to the requirements will be made in the SRS.
- **Result Validation:** The simulation in test NFR-SIM-1 will compare every aggregated result with a reference implementation of the aggregation algorithm written by a team member who did not write the system's version. The worked examples in the SRS will be included, so that the results can also be checked by hand.
- **Requirements Review after Validation:** After the Rev-0 demonstration, the team will review the SRS with the supervisor, as in the SRS verification above, to confirm that the requirements still describe what requesters and providers need, and update the SRS and this plan if they do not.

5 System Tests

This section describes the tests that will be performed to verify that the entire integrated system meets the functional and non-functional requirements specified in the SRS. The system test cases below are described by the following characteristics:

- Control: Is this test manual or automatic?
- Initial State: What is the state of the system before the test is performed?
- Input: What is the input to the system for this test?
- Output: What is the expected output of the system for this test?
- Test Case Derivation: What FRs or NFRs does this test case cover?
- How test will be performed: What is the abstract procedure for performing the test?

5.1 Tests for Functional Requirements

5.1.1 Accounts and Authentication

These tests verify the account authentication requirements in the SRS, including authentication with a compatible Web3 wallet extension and an embedded wallet.

Authentication Using a Web3 Wallet Extension

1. ST-AUTH-EXT-1

   Control: Manual

   Initial State: The application is running in a supported browser on its public page. A compatible Web3 wallet extension is installed and active. Test profiles `EXT-USER-00`, `EXT-USER-01`, and `EXT-USER-02` are associated with wallet fixtures `EXT-WALLET-00`, `EXT-WALLET-01`, and `EXT-WALLET-02`; neither user is signed in.

   Input:

   1. Connect `EXT-WALLET-00` and approve the verification-message signature requested by the application.
   2. Connect `EXT-WALLET-01` and approve its verification-message signature.
   3. Connect `EXT-WALLET-02` and reject the signature request.

   Output:

   1. `EXT-WALLET-00` authenticates as `EXT-USER-00`, whose profile and authenticated features are accessible.

2. `EXT-WALLET-01` authenticates as `EXT-USER-01`, whose profile and authenticated features are accessible.
3. Rejecting the signature leaves the user unauthenticated, with no profile or authenticated features accessible.

Test Case Derivation: FR1 requires the authentication response to verify ownership of the connected wallet address.

How test will be performed: For each wallet, start from a signed-out session, connect the specified extension account, and approve the verification-message signature request. Confirm that `EXT-WALLET-00` opens `EXT-USER-00`'s profile and `EXT-WALLET-01` opens `EXT-USER-01`'s profile. Confirm that the user is granted access to the authenticated features. Repeat the process and reject the signature request; confirm that the user remains signed out and cannot access authenticated features.

2. ST-AUTH-EXT-2

Control: Manual

Initial State: The application is running on its public page in a supported browser. `EXT-WALLET-03` is available in a compatible browser extension and is not yet associated with a DAIIM user profile. The wallet owner is not signed in.

Input:

1. Connect `EXT-WALLET-03` and approve the verification-message signature.
2. Sign out of the resulting account.
3. Reconnect `EXT-WALLET-03` and approve the verification-message signature again.

Output:

1. The first approved signature authenticates the user and associates a profile with `EXT-WALLET-03`; the authenticated features are accessible.
2. Signing out ends the authenticated session; the profile and authenticated features are inaccessible.
3. The second approved signature authenticates the user with the same profile associated with `EXT-WALLET-03`; no second profile exists.

Test Case Derivation: FR1 requires the system to verify ownership of the wallet address and associate that verified address with the same user profile on every successful authentication, including after sign-out. The second successful authentication must restore the profile created during the first sign-in.

How test will be performed: Record the number of DAIIM profiles associated with `EXT-WALLET-03`. Connect the wallet and approve the verification-message signature.

Record the displayed profile identifier and confirm that the wallet is associated with that profile. Sign out and verify that the profile and authenticated features are no longer accessible. Sign in again with the same wallet and approve the signature. Verify that the displayed profile identifier matches the recorded identifier and that the number of profiles associated with the wallet has not increased.

Authentication Using an Embedded Wallet

1. ST-AUTH-EMB-1

   Control: Automatic

   Initial State: The application is running in a test environment with the embedded-wallet provider mocked. Identities `new.email@example.test`, `+15555550101`, and social-account subject `social-new-01` have no associated DAIIM account or embedded wallet. Configure the provider mock to return successful authentication and wallet creation for these identities, with distinct addresses `EMB-WALLET-NEW-01`, `EMB-WALLET-NEW-02`, and `EMB-WALLET-NEW-03`, respectively. The malformed email `not-an-email` should be rejected by the system before the provider is called.

   Input:

   1. Submit `new.email@example.test` using the email sign-in method.
   2. Submit `+15555550101` using the phone sign-in method.
   3. Submit social-account subject `social-new-01` using the supported social sign-in method.
   4. Submit malformed email `not-an-email` using the email sign-in method.

   Output:

   1. `new.email@example.test` creates one embedded wallet `EMB-WALLET-NEW-01` and authenticates the associated user profile.
   2. `+15555550101` creates one embedded wallet `EMB-WALLET-NEW-02` and authenticates the associated user profile.
   3. `social-new-01` creates one embedded wallet `EMB-WALLET-NEW-03` and authenticates the associated user profile.
   4. `not-an-email` is rejected; no wallet or authenticated session is created.

   Test Case Derivation: FR2 requires successful creation for supported identity input after authentication, with the public address and confirmation as outputs. The system must reject malformed identity input before wallet creation. FR1 requires the successful wallet authentication to identify the user.

How test will be performed: For each valid test identity, call the corresponding embedded-wallet sign-in method with the specified email address, phone number, or social-account identity. For each successful case, compare the displayed address with the configured mock address, verify that the corresponding profile is authenticated, and confirm that exactly one wallet was created. For the malformed email, enter it using the email sign-in method and verify that the system reports an input error, does not call the embedded-wallet provider, and does not create a wallet or authenticated session.

2. ST-AUTH-EMB-2

Control: Automatic

Initial State: The application is running in a test environment, and existing accounts `EMB-RETURN-01`, `EMB-RETURN-02`, and `EMB-RETURN-03` are signed out. Their embedded wallet addresses are respectively `EMB-WALLET-RETURN-01`, `EMB-WALLET-RETURN-02`, and `EMB-WALLET-RETURN-03`. Configure the embedded-wallet provider mock to return successful authentication responses with those address mappings. The email `returning.email@example.test` belongs to `EMB-RETURN-01` and `EMB-WALLET-RETURN-01`; the phone number `+15555550102` belongs to `EMB-RETURN-02` and `EMB-WALLET-RETURN-02`; and the social-account subject `social-return-03` belongs to `EMB-RETURN-03` and `EMB-WALLET-RETURN-03`. Configure it to return an authentication failure for `locked.user@example.test`, which has no associated account or wallet.

Input:

1. Sign in as `EMB-RETURN-01`, using wallet `EMB-WALLET-RETURN-01`, with email `returning.email@example.test`.
2. Sign in as `EMB-RETURN-02`, using wallet `EMB-WALLET-RETURN-02`, with phone number `+15555550102`.
3. Provide social-account subject `social-return-03` using the supported social sign-in method for `EMB-RETURN-03`, using wallet `EMB-WALLET-RETURN-03`.
4. Enter `locked.user@example.test` using the email sign-in method; this identity has no associated account or wallet.

Output:

1. `returning.email@example.test` authenticates `EMB-RETURN-01` with its existing profile and `EMB-WALLET-RETURN-01`.
2. `+15555550102` authenticates `EMB-RETURN-02` with its existing profile and `EMB-WALLET-RETURN-02`.
3. `social-return-03` authenticates `EMB-RETURN-03` with its existing profile and `EMB-WALLET-RETURN-03`.

4. `locked.user@example.test` remains unauthenticated, with no profile access or new account.

Test Case Derivation: FR1 requires each verified wallet address to remain associated with the same user profile after sign-out. Each successful response must restore the mapped existing profile rather than create another account. The failed authentication response does not verify wallet ownership, so it must not grant access.

How test will be performed: Call the application's embedded-wallet authentication entry point with the sign-in method and parameters from each returning-user credentials. Configure the provider mock to return that fixture's existing wallet authentication response. Assert that each call returns an authenticated session for the user profile. Then call the same entry point with the sign-in method and identity parameters from the authentication-failure credentials. Assert that the call returns no authenticated session, grants no profile access and displays the appropriate error message.

5.1.2 Inference Task Creation

These tests verify the end-to-end task creation flow, including validation of the task definition, user authorization, and confirmation that the task-creation program instruction succeeds on the blockchain. They also verify that the provider counts, tolerance, and timing fields of a task are required and stored, and that all users, including users who are not signed in, can browse the marketplace and its tasks.

Create and Validate Inference Tasks

1. ST-TASK-CREATE-1

   Control: Automatic

   Initial State: The system is running on a supported blockchain platform. The requester wallet address fixture is authenticated, and the test harness is configured with its wallet signer and sufficient wallet balance to pay transaction fees. The unique task names `Regression-Test-Task`, `Regression-Test-Min-Providers`, `Regression-Test-Invalid-Schema`, `Regression-Test-No-Tolerance`, and `Rejected-Authorization-Task` are not already in use.

   Input:

   1. A valid numerical regression task definition named `Regression-Test-Task`, including a description, numerical input and output schemas, an evaluation metric, a minimum of 2 and a maximum of 4 providers, a reputation tolerance of 2.0, and an acceptance period of 2 minutes.
   2. The valid definition named `Regression-Test-Min-Providers`, with its minimum provider count changed to 1.

3. The valid definition named `Regression-Test-Invalid-Schema`, with an input or output schema containing non-numerical values invalid with a numerical regression task.
4. The valid definition named `Regression-Test-No-Tolerance`, with the reputation tolerance left out.
5. The valid definition named `Rejected-Authorization-Task`.

Output:

1. The valid definition is created with the submitted values, and its authorized transaction succeeds. The task's on-chain account stores a minimum of 2 and a maximum of 4 providers, a reputation tolerance of 2.0, and an acceptance period of 2 minutes.
2. The definition with fewer than 2 minimum providers is rejected, and no task is created.
3. The definition with an invalid schema is rejected, and no task is created.
4. The definition without a reputation tolerance is rejected with an error saying the reputation tolerance is missing, and no task is created.
5. The authorization-rejected definition is not created, and no successful task-creation transaction occurs.

Test Case Derivation: FR6 requires a task name, description, type, input schema, and output schema; the invalid-schema case checks that the definition matches the numerical regression task. FR8 requires at least 2 providers, so a value of 1 must be rejected. FR7 requires the provider counts to be defined per task, and FR9 requires its reputation tolerance. FR10 requires each task to define its acceptance period. FR16 and FR17 define protocol-wide durations, not task fields; ST-REQ-4 checks their resulting request deadlines and broadcasts. ST-REP-1 checks the use of the reputation tolerance. FR5 permits authenticated users to create tasks, while FR4 requires wallet approval before an on-chain state change. Therefore, the valid task should be created only after approval, and rejecting authorization should leave it uncreated. FR61 and FR62 require a final transaction status and a transaction identifier that can be used to verify the successful instruction.

How test will be performed: Submit the valid definition to the task-creation flow and assert that the confirmation details match the submitted definition. Configure the test wallet signer to approve the authorization request. After confirmation, query the public blockchain network using the returned transaction identifier and assert that the transaction succeeded and contains the task-creation instruction. Query the application to confirm that `Regression-Test-Task` is available with the submitted definition, and read its task account on the public blockchain network to compare each provider count, tolerance, and timing field with the submitted value. Submit the minimum-provider, invalid-schema, missing-tolerance, and missing-maximum-time definitions;

assert that each is rejected before authorization and no task is created. Finally, submit the valid definition named `Rejected-Authorization-Task`, configure the signer to reject authorization, and assert that no task-creation instruction is submitted and no task is created.

Browsing Tasks and the Marketplace

1. ST-TASK-BROWSE-1

   Control: Automatic

   Initial State: The system is running on the public blockchain network. `Regression-Test-Task` from ST-TASK-CREATE-1 exists, with a required minimum provider stake of 100 tokens, and `PROV-41` and `PROV-42` are registered for it with reputation scores of 50 and 70. `PROV-43` is an authenticated provider that is not registered for `Regression-Test-Task`. A visitor `V-01` opens the web application without signing in. Requester `RQ-41` is signed in on another browser.

   Input:

   1. `V-01` opens the marketplace page.
   2. `V-01` opens the list of inference tasks.
   3. `V-01` opens `Regression-Test-Task`.
   4. `PROV-43` registers for `Regression-Test-Task`, and `V-01` reloads `Regression-Test-Task`.
   5. `RQ-41` opens the marketplace page, the task list, and `Regression-Test-Task`.
   6. `V-01` tries to create a task and to submit an inference request for `Regression-Test-Task`.

   Output:

   1. `V-01` sees the registered providers, the models they offer, and their reputation scores, including `PROV-41` (50) and `PROV-42` (70).
   2. The list includes `Regression-Test-Task`.
   3. `V-01` sees `Regression-Test-Task`'s full definition (name, description, type, input and output schemas, evaluation metric, number of providers, reputation tolerance, and acceptance period), a required minimum provider stake of 100 tokens, and 2 registered providers.
   4. `Regression-Test-Task` now shows 3 registered providers.
   5. `RQ-41` sees the same information as `V-01`.
   6. `V-01` is asked to sign in, and no task or request is created.

Test Case Derivation: FR3 lets unauthenticated users browse marketplace information, including registered providers, available models, and provider reputations. FR11 lets all users, signed in or not, browse tasks and view each task's definition, required minimum provider stake, and number of registered providers, and the count must change when a provider registers. FR5 and FR18 only allow authenticated users to create tasks and submit requests, so browsing access must not extend to those actions.

How test will be performed: Run the steps in an automated browser test, using a new browser profile for `V-01` with no wallet connected. Compare the values shown on each page with the task and provider accounts on the public blockchain network. For step 6, check that no signing request is sent and that no new task or request account is created.

5.1.3 Provider Registration and Staking

These tests verify the provider registration and staking requirements in the SRS. They cover registering as an inference provider for a task, locking the required minimum stake in escrow, unregistering when eligible, and viewing provider information through the dashboard.

Registering as an Inference Provider

1. ST-PROV-1

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and connected to a compatible wallet. The provider is not registered for inference task `TASK-01`. The task requires a minimum stake of tokens, and the provider's wallet contains sufficient tokens to complete the transaction.

   Input:

   1. Navigate to `TASK-01` and select the option to register as an inference provider.
   2. Review the required minimum stake of tokens and the associated registration details.
   3. Confirm the registration and approve the wallet transaction.

   Output:

   1. The provider is successfully registered for `TASK-01`.
   2. Exactly the minimum amount of tokens are locked in escrow as the provider's stake.
   3. The provider is added to the task's list of registered providers and receives the base reputation score for the task.

4. The system displays the transaction status and, once confirmed, the transaction identifier.

Test Case Derivation: FR12 requires authenticated users to be able to register as inference providers by staking the required minimum amount of tokens. FR13 requires the minimum stake to be locked in escrow. The transaction authorization and confirmation behaviour is also covered by the related transaction requirements.

How test will be performed: Authenticate a provider account and navigate to `TASK-01`. Select the registration option and verify that the system displays the required minimum stake. Confirm the registration and approve the transaction through the connected wallet. After the transaction is confirmed, inspect the task's registered-provider list and the provider's account information. Verify that the provider is registered, has received the base reputation score, and that exactly the minimum amount of tokens have been transferred into escrow.

2. ST-PROV-2

Control: Manual

Initial State: The application is running in a supported browser. A provider account is authenticated and connected to a compatible wallet. The provider is not registered for `TASK-02`. The task requires a minimum stake of `MINIMUM_PROVIDER_STAKE` tokens, but the provider's wallet contains `MINIMUM_PROVIDER_STAKE` - 1 tokens.

Input:

1. Navigate to `TASK-02` and select the option to register as an inference provider.
2. Attempt to proceed with registration despite having an insufficient token balance.

Output:

1. The system prevents the provider from completing registration.
2. An error message indicates that the provider's token balance is insufficient and identifies the required shortfall of one token.
3. No stake is transferred into escrow, and the provider is not added to the task's list of registered providers.

Test Case Derivation: FR12 and FR13 require the provider to stake `MINIMUM_PROVIDER_STAKE` tokens to register. The related balance-check assumptions require the system to verify that sufficient funds are available before processing the transaction.

How test will be performed: Authenticate a provider account whose wallet contains `MINIMUM_PROVIDER_STAKE` - 1 tokens and navigate to `TASK-02`. Attempt to register for the task. Verify that the system displays an insufficient-balance error indicating a shortfall of one token, that no transaction transfers tokens into escrow, and that the provider does not appear in the task's registered-provider list.

Unregistering from an Inference Task

1. ST-PROV-3

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and registered for `TASK-03`. The provider has no active accepted inference requests for this task. The provider's current reputation score is at or above the task's reduced-stake-return threshold, and the provider has a remaining stake of tokens.

   Input:

   1. Navigate to the provider's registration details for `TASK-03`.
   2. Select the option to unregister from the task.
   3. Review and confirm the unregistration.
   4. Approve the wallet transaction, if prompted.

   Output:

   1. The provider is successfully unregistered from `TASK-03`.
   2. The provider is removed from the task's list of registered providers.
   3. The provider's entire remaining stake of tokens is returned to their token account.
   4. The system displays the transaction status and, once confirmed, the transaction identifier.

   Test Case Derivation: FR14 requires the system to allow providers to unregister when they have no active accepted inference requests. FR59 requires the entire remaining stake to be returned when the provider's reputation is at or above the task's reduced-stake-return threshold.

   How test will be performed: Authenticate the registered provider and verify that they have no active accepted inference requests for `TASK-03`. Select the unregistration option and confirm the action. After the transaction is confirmed, inspect the provider's registration status, the task's registered-provider list, and the provider's token balance. Verify that the provider has been removed from the list and that the entire remaining stake has been returned.

2. ST-PROV-4

   Control: Manual

Initial State: The application is running in a supported browser. A provider account is authenticated and registered for `TASK-04`. The provider has accepted an inference request that is still active for this task.

Input:

1. Navigate to the provider's registration details for `TASK-04`.
2. Select the option to unregister from the task.
3. Attempt to confirm the unregistration.

Output:

1. The system rejects the unregistration attempt because the provider has an active accepted inference request.
2. An error message explains why the provider cannot unregister.
3. The provider remains registered for `TASK-04`, and their remaining stake is unchanged.

Test Case Derivation: FR14 requires that a provider must not have accepted any active inference requests for the task before they can unregister.

How test will be performed: Authenticate a provider account that is registered for `TASK-04` and has an active accepted inference request. Attempt to unregister from the task. Verify that the system rejects the attempt, displays an appropriate error message, maintains the provider's registration, and does not modify the provider's remaining stake.

Viewing the Provider Dashboard

1. ST-PROV-5

Control: Manual

Initial State: The application is running in a supported browser. A provider account is authenticated and registered for two inference tasks, `TASK-05` and `TASK-06`. The provider has an existing reputation score, staked token balance, and earned rewards recorded for each task.

Input:

1. Navigate to the provider dashboard.
2. View the provider's information for `TASK-05` and `TASK-06`.

Output:

1. The dashboard displays the provider's reputation score, staked tokens, and earned rewards for `TASK-05`.
2. The dashboard displays the provider's reputation score, staked tokens, and earned rewards for `TASK-06`.
3. The displayed values correspond to the provider's recorded information for each task.

Test Case Derivation: FR15 requires inference providers to be able to view their reputation score, staked tokens, and rewards earned for each inference task they are registered for.

How test will be performed: Authenticate the provider account and navigate to the provider dashboard. Retrieve the provider's recorded reputation, stake, and reward values for both tasks from the system's records. Compare these values with the information displayed on the dashboard and verify that the correct information is displayed for each task.

5.1.4 Inference Requests

These tests verify the inference request requirements in the SRS. They cover submitting requests for existing inference tasks, validating request input data against the task's input schema, authorizing payments into escrow, and broadcasting submitted requests to registered providers.

Submitting an Inference Request

1. ST-REQ-1

   Control: Manual

   Initial State: The application is running in a supported browser. A requester account is authenticated and connected to a compatible wallet. An existing inference task, `TASK-01`, is available and has at least two registered providers. The task's input schema requires a numerical input field named `input_value`. The requester's wallet contains sufficient tokens and tokens to submit the request.

   Input:

   1. Navigate to `TASK-01` and select the option to submit an inference request.
   2. Provide valid input data containing `input_value` with a numerical value
   3. Review the request details and confirm the submission.
   4. Authorize the payment through the connected wallet.

   Output:

1. The system accepts the input data and successfully submits the inference request for `TASK-01`.
2. The request is assigned a unique identifier and its submission status is displayed to the requester.
3. The input data is stored securely for processing and is not recorded directly on-chain.
4. The requester is informed that the request has been submitted and is awaiting provider participation.

Test Case Derivation: FR18 requires authenticated users to be able to submit inference requests for existing tasks, with input data conforming to the task's input schema. The related input-data requirements specify that input data must be stored securely and not placed directly on-chain.

How test will be performed: Authenticate a requester account and navigate to `TASK-01`. Enter the valid input data and submit the request. Authorize the transaction through the connected wallet. Verify that the system creates a request with a unique identifier, displays the appropriate submission status, and securely stores the input data. Inspect the on-chain request record to confirm that the input data itself has not been recorded on-chain.

2. ST-REQ-2

Control: Manual

Initial State: The application is running in a supported browser. A requester account is authenticated and connected to a compatible wallet. An existing inference task, `TASK-02`, is available and has at least two registered providers. The task's input schema requires a numerical field named `input_value`.

Input:

1. Navigate to `TASK-02` and select the option to submit an inference request.
2. Provide input data with the required `input_value` field missing.
3. Repeat the submission attempt with `input_value` provided as a non-numerical value.

Output:

1. The system rejects both invalid input submissions.
2. The system displays an error message identifying the missing or invalid input field.
3. No inference request is submitted, no request identifier is created, and no payment is transferred into escrow.

Test Case Derivation: FR19 requires the system to reject inference requests whose input data does not conform to the task's input schema. FR18 requires submitted requests to include input data that conforms to that schema.

How test will be performed: Authenticate a requester account and navigate to `TASK-02`. Attempt to submit a request with the required input field missing. Verify that the system rejects the request and identifies the missing field. Repeat the test using a non-numerical value for `input_value`. Verify that the request is rejected again and that neither attempt creates a request or transfers funds into escrow.

Request Payment and Authorization

1. ST-REQ-3

Control: Manual

Initial State: The application is running in a supported browser. A requester account is authenticated and connected to a compatible wallet. An existing inference task, `TASK-03`, is available and has at least two registered providers.The requester's wallet contains sufficient tokens to authorize a payment.

Input:

1. Navigate to `TASK-03` and provide valid input data.
2. Select a payment amount greater than `MINIMUM_REQUEST_PAYMENT`, review the transaction details, and confirm the request submission.
3. Approve the transaction through the connected wallet.
4. Submit another request using a payment amount equal to `MINIMUM_REQUEST_PAYMENT`.
5. Submit another request using a payment amount below `MINIMUM_REQUEST_PAYMENT`.

Output:

1. The system accepts the valid request input.
2. The system accepts the payment amount because it exceeds the task's minimum payment requirement and prompts the requester to authorize a wallet transaction transferring the tokens into escrow.
3. After authorization and transaction confirmation, exactly the authorized amount of tokens is transferred into escrow. The system displays the transaction status and, once confirmed, the transaction identifier.
4. The system rejects the payment amount because it is equal to the minimum payment requirement. The requester is not prompted to authorize a wallet transaction, and no tokens are transferred into escrow.

5. The system rejects the payment amount because it is below the minimum payment requirement. The requester is not prompted to authorize a wallet transaction, and no tokens are transferred into escrow.

Test Case Derivation: FR20 requires the requester to choose a token payment greater than the minimum payment requirement and authorize a wallet transaction transferring that amount into escrow. Therefore, payments equal to or below the minimum must be rejected. The related transaction authorization and preview requirements ensure that the requester can review and authorize the transaction. FR.SA.8 requires the amount charged to match the amount authorized.

How test will be performed: Authenticate a requester account and navigate to `TASK-03`. Enter valid request input and first select a payment amount greater than the minimum defined for the task. Review the transaction preview and approve the transaction through the connected wallet. After confirmation, inspect the requester's token balance and the escrow account's balance to verify that exactly the authorized amount has been transferred.

Next, submit a request using a payment amount equal to the minimum defined for the task, followed by another request using an amount below the minimum. For both attempts, verify that the system rejects the payment before requesting wallet authorization and that no tokens are transferred into escrow.

Broadcasting Submitted Inference Requests

1. ST-REQ-4

Control: Automatic

Initial State: The system is running in a test environment. An existing inference task, `TASK-04`, has three registered providers: `PROV-01`, `PROV-02`, and `PROV-03`. All three providers are available to receive inference requests. The task has a configured acceptance period of 10 minutes. The test protocol sets COMMIT_PERIOD and REVEAL_PERIOD to 30 minutes each.

Input:

1. Submit a valid inference request for `TASK-04`.
2. Set the proposed payment amount to a number of tokens.

Output:

1. The system successfully submits the inference request and broadcasts it to all three registered providers.

2. Each provider receives the proposed payment amount, task definition, acceptance-period expiry time, commit deadline, and latest possible reveal deadline. At commit-phase end, the actual reveal deadline is also broadcast. The broadcast information is consistent across all three providers and matches the submitted request and task definition.

Test Case Derivation: FR21 requires the system to broadcast the task definition, proposed payment amount, acceptance-period expiry time, and the calculated commit and reveal deadlines to all providers registered for the task after an inference request is submitted.

How test will be performed: Configure the test protocol with COMMIT_PERIOD and REVEAL_PERIOD set to 30 minutes each. Configure `TASK-04` with the three specified registered providers and a 10-minute acceptance period. Submit a valid inference request with the specified payment amount. Monitor the messages or notifications received by each provider. Verify that the request is successfully submitted and broadcast to all three providers. Verify that each provider receives the correct task definition, proposed payment amount, acceptance-period expiry time, commit deadline, and latest possible reveal deadline. Confirm that the commit deadline is 10 minutes after acceptance ends, the latest reveal deadline is 30 minutes after the commit deadline, and an actual reveal deadline is broadcast when the commit phase ends.

5.1.5 Provider Acceptance and Predictions

These tests verify the provider acceptance and prediction requirements in the SRS. They cover accepting or declining inference requests during the acceptance period, reserving stake for accepted requests, submitting predictions, and cancelling requests that receive too few valid predictions.

Accepting an Inference Request

1. ST-ACC-1

   Control: Automatic

   Initial State: The system is running on a supported blockchain platform. `TASK-07` requires at least 2 accepting providers, allows at most 3, and has an acceptance period of 2 minutes. `PROV-11`, `PROV-12`, and `PROV-13` are registered for `TASK-07`. A requester has submitted inference request `REQ-01` for `TASK-07`.

   Input:

   1. `PROV-11` and `PROV-12` accept `REQ-01` and sign the acceptance in their wallets.
   2. `PROV-13` declines `REQ-01`.
   3. The acceptance period of `REQ-01` elapses.

Output:

1. PROV-11 and PROV-12 are recorded as accepting providers, and part of each one's stake is reserved for REQ-01 without any tokens being transferred.
2. The decline from PROV-13 is recorded, and no stake is reserved for it.
3. The acceptance period, which started when REQ-01 was broadcast, ends 2 minutes later, and REQ-01 moves to its commit phase with PROV-11 and PROV-12.

Test Case Derivation: FR22 starts the acceptance period at broadcast and lets registered providers accept or decline, and FR10 sets its length. FR4 requires each acceptance to be signed, and FR33 requires part of the provider's stake to be reserved for the request.

How test will be performed: Send the two acceptances and the decline. Query REQ-01 and the provider accounts on the blockchain, and verify the acceptance period times, the recorded acceptances and decline, and the reservations. After the period ends, verify that REQ-01 is in its commit phase.

2. ST-ACC-2

Control: Automatic

Initial State: The system is running on a supported blockchain platform. PROV-11 to PROV-15 are registered for TASK-07, and PROV-16 is not. A requester has submitted inference requests REQ-02 and REQ-03 for TASK-07.

Input:

1. PROV-16 tries to accept REQ-02.
2. After the acceptance period of REQ-02 ends, PROV-14 tries to accept it.
3. PROV-11, PROV-12, and PROV-13 accept REQ-03, then PROV-15 tries to accept it.

Output:

1. The acceptance from PROV-16 is rejected with a not-registered error.
2. The acceptance from PROV-14 is rejected because the acceptance period has ended.
3. The acceptance from PROV-11, PROV-12, and PROV-13 are registered. The acceptance from PROV-15 is rejected because REQ-03 already has 3 accepting providers.

Test Case Derivation: FR22 only lets registered providers accept a request. FR23 ignores acceptances after the acceptance period ends, and FR7 sets the task's maximum number of accepting providers for each request.

How test will be performed: Send each acceptance at the stated time, using the on-chain clock to wait for the end of the acceptance period of REQ-02. Verify that each faulty provider is rejected with the expected error and that none of these providers is recorded as accepting.

3. ST-ACC-3

   Control: Automatic

   Initial State: The system is running on a supported blockchain platform. PROV-11 is registered for TASK-07, which requires at least 2 accepting providers. A requester has submitted inference request REQ-04 for TASK-07 with a payment of REQUEST_PAYMENT.

   Input:

   1. Only PROV-11 accepts REQ-04 before the acceptance period ends.

   Output:

   1. REQ-04 is cancelled when the acceptance period ends, and does not enter a commit phase. The REQUEST_PAYMENT is returned to the requester, and the stake reserved from PROV-11 is released.

   Test Case Derivation: FR24 cancels a request that fewer than the task's minimum number of providers have accepted, and returns the requester's payment. FR34 releases the reservations of a cancelled request.

   How test will be performed: Have PROV-11 accept REQ-04, and wait until the on-chain clock passes the end of the acceptance period. Verify that REQ-04 is cancelled, that the requester's wallet balance went back up by REQUEST_PAYMENT, and that PROV-11 has no reservation left for REQ-04.

4. ST-ACC-4

   Control: Automatic

   Initial State: The system is running on a supported blockchain platform. Each request for TASK-08 reserves TASK_RESERVATION_AMOUNT of an accepting provider's stake. PROV-17 is registered for TASK-08 with a stake of MINIMUM_PROVIDER_STAKE + TASK_RESERVATION_AMOUNT $\times$ 2.5. A requester has submitted inference requests REQ-05, REQ-06, REQ-07, and REQ-08.

   Input:

   1. PROV-17 accepts REQ-05, REQ-06, and REQ-07 while all three are open.
   2. After REQ-05 completes, PROV-17 accepts REQ-08.

   Output:

   1. The acceptances of REQ-05 and REQ-06 are recorded. The acceptance of REQ-07 is rejected, because a third reservation would take the stake of PROV-17 below MINIMUM_PROVIDER_STAKE.

2. After REQ-05 completes, its reservation is released and the acceptance of REQ-08 is recorded.

Test Case Derivation: FR31 rejects an acceptance if the stake left after all reservations would be below MINIMUM_PROVIDER_STAKE. With TASK_RESERVATION_AMOUNT$\times 2.5$ above the minimum, two TASK_RESERVATION_AMOUNT reservations fit and a third does not. FR32 releases a reservation when its request ends.

How test will be performed: Have PROV-17 accept the requests in order, and read its reservations from the on-chain program accounts after each acceptance. Run REQ-05 to completion before the last acceptance, and verify that each acceptance is recorded or rejected as expected.

Submitting Predictions

1. ST-PRED-1: Commit and reveal predictions

Control: Automatic

Initial State: The system is running on a supported blockchain platform. TASK-09 and TASK-10 have numerical output schemas. In this test configuration, the protocol-wide COMMIT_PERIOD and REVEAL_PERIOD are 5 minutes each. For inference request REQ-09, PROV-18 and PROV-19 are accepted providers, and its acceptance period has ended. For inference request REQ-10, PROV-18 to PROV-22 are accepted providers, while PROV-23 is registered for TASK-10 but did not accept REQ-10.

Input:

1. For inference request REQ-09, PROV-18 and PROV-19 each generate and persist a unique 256-bit salt, hash the request ID, provider wallet, prediction, and salt, and submit the commitment during the commit phase. Both reveal their prediction and salt during the reveal phase.
2. During the acceptance period for REQ-10, PROV-18 attempts to submit a commitment.
3. During the commit phase for REQ-10, PROV-18 commits to 10.0, PROV-19 commits to a valid numerical prediction, PROV-20 commits to a string prediction, and PROV-21 commits to a valid numerical prediction. PROV-22 does not commit, and PROV-23 attempts to commit despite not accepting the request.
4. After the commit deadline for REQ-10, PROV-22 attempts to submit a commitment.
5. During the reveal phase for REQ-10, PROV-18 reveals 10.0 and then attempts a second reveal of 50.0. PROV-19 reveals with a salt that does not match its commitment. PROV-20 reveals the string prediction with the matching salt. PROV-21 does not reveal before the reveal deadline.

6. After the reveal deadline for REQ-10, PROV-21 attempts to reveal its prediction and salt.

Output:

1. Both REQ-09 commitments and reveals are accepted. The commit phase ends after both providers commit, and the reveal phase ends after both valid reveals, before their deadlines.
2. The early commitment from PROV-18 is rejected because the commit phase has not started.
3. The commitments from PROV-18 to PROV-21 are accepted. The commitment from PROV-23 is rejected because it did not accept REQ-10.
4. The late commitment from PROV-22 is rejected because the commit phase has ended.
5. The first valid reveal from PROV-18 is accepted and its second reveal is rejected. The mismatched reveal from PROV-19 and non-conforming reveal from PROV-20 are rejected and do not count toward the minimum. PROV-21 has no reveal recorded before the deadline.
6. The late reveal from PROV-21 is rejected because the reveal phase has ended.

Test Case Derivation: FR16 and FR17 set the phase durations for every request. FR28 requires commitments to bind the request ID, provider wallet, prediction, and salt. FR30 requires the program to verify the commitment and output schema. FR29 limits each provider to one commitment and one reveal. FR27 and FR31 define the phase timing and reject submissions outside their phases.

How test will be performed: Run both requests using the on-chain clock. Verify the salts contain at least 256 bits and remain available in the SDK until reveal. Recompute each commitment from the request ID, provider wallet, prediction, and salt, and compare it with the stored commitment. Check the recorded phase end times and verify every acceptance or rejection listed above.

2. ST-PRED-3

Control: Automatic

Initial State: The system is running on a supported blockchain platform. TASK-10 requires 3 valid predictions and allows at most 5 accepting providers. PROV-23 to PROV-27 have accepted inference request REQ-11 for TASK-10, which has a payment of REQUEST_PAYMENT, and the per-provider compensation is MINIMUM_REQUEST_PAYMENT.

Input:

1. PROV-23 and PROV-24 submit valid commitments and reveals.

2. PROV-25 commits to a non-conforming prediction and reveals it with the correct salt.
3. PROV-26 does not submit a commitment.
4. PROV-27 submits a commitment but does not reveal.
5. The commit and reveal deadlines elapse.

Output:

1. Both predictions are accepted, and once REQ-11 is cancelled, PROV-23 and PROV-24 each receive MINIMUM_REQUEST_PAYMENT of compensation.
2. The reveal from PROV-25 is rejected and does not count toward the 3 valid predictions. Its stake is slashed by SLASH_FRACTION, reflected in the remaining reservation.
3. The stake of PROV-26 is slashed by SLASH_FRACTION under FR54, including because the request is cancelled.
4. The stake of PROV-27 is slashed by SLASH_FRACTION under FR55, including because the request is cancelled.
5. REQ-11 is cancelled with no aggregated result, the requester gets the REQUEST_PAYMENT back, and all remaining reservations for REQ-11 are released.

Test Case Derivation: FR32 cancels a request with fewer valid predictions than the task requires and returns the payment, and under FR30 the rejected prediction does not count. FR35 pays each provider with a valid prediction the per-provider compensation from the providers without one, and FR34 releases the remaining reservations.

How test will be performed: Send the predictions and wait until the on-chain clock passes the commit and reveal deadlines. Verify that REQ-11 is cancelled, and compare the wallet balances and reservations on the blockchain with the expected outputs.

5.1.6 Aggregation and Results

These tests verify the aggregation and results requirements in the SRS. They cover combining valid predictions into a final result with the weighted-median algorithm in Appendix A of the SRS, returning the result and its details to the requester, and letting requesters follow the status of their requests. Each numbered input in ST-AGG-1 is a separate request whose predictions are chosen so that the expected result can be worked out by hand.

Aggregating Predictions

1. ST-AGG-1

Control: Automatic

Initial State: The system is running on the public blockchain network. TASK-11 has a numerical output schema and requires at least 3 valid predictions. PROV-31 to PROV-36 are registered for TASK-11 with these reputation scores: PROV-31: 40, PROV-32: 60, PROV-33: 50, PROV-34: 50, PROV-35: 50, PROV-36: 0. PROV-36's score of 0 is set directly by the test fixture, since a provider whose reputation reaches 0 is normally unregistered. Requester RQ-01 submits each request below, and the listed providers accept it.

Input:

1. For REQ-21, PROV-31 to PROV-35 submit 8.5, 11, 12, 13, and 20.
2. For REQ-22, PROV-33, PROV-34, PROV-35, and a fourth provider with a reputation of 50 submit 10, 12, 14, and 18.
3. For REQ-23, PROV-31, PROV-32, and PROV-33 submit 10, 20, and 30, PROV-36 submits 1000, and PROV-34 submits a string instead of a number.
4. For REQ-24, four providers with equal reputation submit 1.0001, 1.0001, 1.0002, and 1.0002.
5. For REQ-25, four providers with equal reputation submit -1.0002, -1.0002, -1.0001, and -1.0001.

Output:

1. The result of REQ-21 is 12, with 5 included predictions, a minimum of 8.5, and a maximum of 20.
2. The result of REQ-22 is 13, with 4 included predictions, a minimum of 10, and a maximum of 18.
3. The result of REQ-23 is 20, with 3 included predictions, a minimum of 10, and a maximum of 30. The predictions from PROV-36 and PROV-34 are not included.
4. The result of REQ-24 is 1.0002.
5. The result of REQ-25 is -1.0002.

For every request, RQ-01 is shown the result, the number of included predictions, the minimum and maximum included predictions, and the time elapsed since the request was submitted.

Test Case Derivation: FR36 weights each included prediction by its provider's reputation divided by the total reputation of the included providers, and takes the weighted median from Appendix A of the SRS.

1. The weights are 0.16, 0.24, 0.2, 0.2, and 0.2. The running total first goes above 0.5 at 12.
2. Each weight is 0.25. The running total is exactly 0.5 after 12, so the result is the average of 12 and 14.

3. FR36 leaves out `PROV-36` (reputation 0) and `PROV-34` (rejected under FR30). The remaining weights are 0.27, 0.4, and 0.33, and the running total first goes above 0.5 at 20. Leaving out 1000 means it is not the maximum.
4. The running total is exactly 0.5 after the second 1.0001, so the result is 1.00015, which Appendix A rounds away from zero to 1.0002.
5. The same case with negative values, which rounds to −1.0002.

FR37 requires the result, the elapsed time, and the number of included predictions to be returned to the requester, and FR38 requires the minimum and maximum of the included predictions.

How test will be performed: Run each request until its reveal phase ends. Read the stored result from the request's on-chain account and the result details shown to `RQ-01` in the application, and compare both with the expected outputs. Check that the elapsed time shown is within 1 second of the time between the request's submission and its result on the blockchain.

Viewing Results

1. ST-AGG-2

   Control: Automatic

   Initial State: The system is running on the public blockchain network. Requester `RQ-02` has submitted these requests for `TASK-11`:

   - `REQ-26`, which is in its acceptance period.
   - `REQ-27`, which is in its commit phase.
   - `REQ-28`, which has finished, with every reward transfer successful.
   - `REQ-29`, which was cancelled because too few providers accepted it.
   - `REQ-30`, which has a final result, but one provider's reward transfer failed because its token account was closed.

   Requester `RQ-03` has submitted no requests.

   Input:

   1. `RQ-02` opens their request history.
   2. `RQ-02` opens `REQ-28`.
   3. `RQ-02` opens `REQ-30`.
   4. `RQ-02` opens `REQ-26` and `REQ-29`.
   5. `RQ-03` opens their request history.

Output:

1. The history lists `REQ-26` to `REQ-30`, each with its current status: accepting providers (`REQ-26`), collecting predictions (`REQ-27`), Completed (`REQ-28`), Cancelled (`REQ-29`), and Pending Transfer (`REQ-30`).
2. `REQ-28` shows its final result, the number of included predictions, the minimum and maximum included predictions, and the elapsed time.
3. `REQ-30` shows its final result and the same details, even though its status is Pending Transfer.
4. `REQ-26` and `REQ-29` show their status and no result.
5. `RQ-03`'s history is empty and does not show any of `RQ-02`'s requests.

Test Case Derivation: FR39 requires requesters to see the status of each request they submitted and, once available, its result. FR37 and FR38 list the details returned with a result. FR47 says a failed reward transfer must not affect the requester's access to the result, so `REQ-30` still shows it. FR51 only marks a request Completed once every transfer has succeeded or been forfeited, which is why `REQ-30` is Pending Transfer and `REQ-28` is Completed. FR24 cancels a request that too few providers accepted, which gives `REQ-29` its status and no result.

How test will be performed: Set up the five requests by running them to the stated points, closing one provider's token account before rewards are paid for `REQ-30`. Sign in as `RQ-02` and `RQ-03` in an automated browser test, open each page, and compare what is shown with the expected outputs and with the request accounts on the public blockchain network.

5.1.7 Reputation, Rewards, and Penalties

These tests verify the reputation, reward, reward transfer, and penalty requirements (FR40–FR59) and the safety requirements that protect reward and penalty transfers (FR.SA.9, FR.SA.10). Each test follows a scenario that a group of providers would go through on the real system: completing a request, having a reward transfer fail, misbehaving during a request, and leaving a task. The tests run on a local validator for the public blockchain network or the public blockchain network, with mock inference providers that submit preset predictions, so that the expected results can be worked out by hand before the test is run. Boundary cases for each rule (such as a distance exactly equal to $\bar{d}$) are covered by the unit tests.

The values of `SLASH_FRACTION`, `MAX_REPUTATION_PENALTY`, `MINIMUM_PROVIDER_STAKE`, `REPEAT_OFFENSE_COUNT`, `REPEAT_OFFENSE_WINDOW`, and `PENDING_TRANSFER_PERIOD` are defined in the SRS. The tests read them from the deployed program's configuration, so they do not need to change if the values change.

1. ST-REP-1: Completed request updates reputations and pays rewards

Control: Automatic

Initial State: A numerical regression task `Rep-Test-Task` exists with a reputation tolerance $T = 2.0$. Mock providers `P1` to `P5` are registered with reputation scores of 40, 60, 50, 50, and 50. A requester has submitted an inference request with a payment of 1000 tokens held in escrow, and all five providers have accepted it.

Input:

1. The providers submit the valid predictions `P1`: 8.5, `P2`: 11, `P3`: 12, `P4`: 13, `P5`: 20, and the request runs to completion.

Output:

1. The final aggregated result is 12. The distances from the result are $d = (3.5, 1, 0, 1, 8)$, so $\bar{d} = 2.7$ and $\bar{d} + T = 4.7$.
2. `P2`, `P3`, and `P4` ($d_i \leq 2.7$) have their reputation increased. `P1` ($2.7 < 3.5 \leq 4.7$) has an unchanged reputation. `P5` ($8 > 4.7$) has its reputation decreased.
3. `P5` receives no reward. `P1` to `P4` receive reward transfers from the request's escrow address of 200, 300, 250, and 250 tokens, in proportion to their reputation scores (40, 60, 50, 50).
4. The reward transfers add up to no more than 1000 tokens, the request's status is Completed, and the backend log has a provider status check for each provider recorded before that provider's reward transfer.

Test Case Derivation: FR40 requires d_i and $\bar{d}$ to be computed for every included provider, and FR42, FR43, and FR44 give the three possible outcomes; this fixture places at least one provider in each. FR46 limits rewards to providers whose reputation did not decrease, split by reputation, so `P1` still qualifies and `P5` does not. FR51 allows the request to be Completed once every reward transfer has succeeded. FR.SA.9 forbids paying out more than the reward, and FR.SA.10 requires each provider's status to be checked before a reward is sent. The expected reward shares use the reputation snapshot taken for aggregation (FR36); if the design uses the scores after the update instead, the expected shares will be recomputed from those scores.

How test will be performed: Record each provider's reputation and token balance, run the request, and read the result, the reputation scores, and the reward transfers from both the application and the on-chain accounts. Assert the values above, allowing up to one base unit of difference per reward for integer rounding, and check the backend log for the status checks.

2. ST-REP-2: A reward transfer fails and is held

Control: Automatic

Initial State: The same task, providers, and request as ST-REP-1, with two changes: `P2`'s token account is closed before rewards are distributed, so it cannot receive tokens, and `P3`'s token account is also closed. The test controls the validator's clock so that time can be moved forward past the pending transfer deadline.

Input:

1. The providers submit the same predictions as ST-REP-1, and the request reaches reward distribution.
2. `P2` opens a new token account and retries its reward transfer from the provider dashboard before the pending transfer deadline.
3. `P3` does nothing, and the clock is moved past the pending transfer deadline (end of the reveal phase plus `PENDING_TRANSFER_PERIOD`).

Output:

1. `P1` and `P4` receive their rewards (200 and 250 tokens). The 300 tokens for `P2` and the 250 tokens for `P3` stay in the request's escrow, both transfers are recorded as unresolved, and the request's status is Pending Transfer. The requester can still see the final aggregated result.
2. `P2` and `P3` each receive a notification, and each of their dashboards shows the held reward amount and the pending transfer deadline.
3. After the retry, `P2`'s new token account holds the 300 tokens and `P2`'s transfer is recorded as resolved. The request stays in Pending Transfer because `P3`'s transfer is still unresolved.
4. After the deadline, `P3`'s 250 tokens are burned (the total burned supply increases by 250), `P3`'s transfer is recorded as forfeited, and the request's status changes to Completed.

Test Case Derivation: FR47 requires a failed transfer to be held in escrow and recorded as unresolved, with the request set to Pending Transfer, while other transfers and the result are not affected. FR48 requires a notification and the dashboard details. FR49 allows the provider to retry before the deadline, and FR50 requires the held reward to be burned at the deadline. FR51 only allows the request to become Completed once every transfer has succeeded or been forfeited, which is why it stays in Pending Transfer after `P2`'s retry.

How test will be performed: After each input step, read the request's status, the escrow balance, each provider's token balance, the transfer records, and the total burned supply from the blockchain, and check each provider's notifications and dashboard in the application.

3. ST-PEN-1: Misbehaving providers in a single request

Control: Automatic

Initial State: Task `Penalty-Test-Task` has a numerical output schema. Providers `P1` to `P5` are registered with 100 tokens of stake each. `P1` has a reputation score of 100, and the others have 50. A request has been submitted and all five providers have accepted it.

Input:

1. `P1` and `P2` submit predictions close to each other (10 and 10.5).
2. `P3` submits a valid prediction far from the others (1,000,000).
3. `P4` submits a prediction that does not match the output schema (a string instead of a number).
4. `P5` submits nothing, and the protocol-wide COMMIT_PERIOD and REVEAL_PERIOD are allowed to pass.

Output:

1. The final aggregated result is 10.25. `P1`'s reputation stays at 100, and `P2`'s reputation increases.
2. `P3`'s reputation decreases by no more than `MAX_REPUTATION_PENALTY`, and its stake stays at 100 tokens.
3. `P4`'s prediction is rejected. `P4` and `P5` each have their stake reduced to $100 \times (1 -$ `SLASH_FRACTION`) tokens and their reputation decreased.
4. `P1`'s and `P2`'s stakes are unchanged, and every reputation score is within $[0, 100]$.

Test Case Derivation: The result of 10.25 follows from the weighted-median algorithm (FR36): `P1`, `P2`, and `P3` are included with weights 0.5, 0.25, and 0.25, and the running total is exactly 0.5 after 10, so the result is the average of 10 and 10.5. FR53 requires slashing and a reputation decrease for a rejected prediction, and FR54, FR55 requires the same for a provider that accepted but did not submit. FR52 allows slashing only in those two cases, so the valid prediction from `P3` must cost reputation and not stake. FR45 caps the reputation decrease from a single request, which the distant prediction from `P3` checks. FR41 keeps scores within $[0, 100]$, which `P1` checks at the top of the range.

How test will be performed: Use a test protocol configuration with short COMMIT_PERIOD and REVEAL_PERIOD values. Record each provider's stake and reputation, run the request until it completes, and read the values again from the on-chain stake and reputation accounts. Assert the expected values for each provider.

4. ST-PEN-2: Providers leaving a task

Control: Automatic

Initial State: Task `Exit-Test-Task` has an exit reputation threshold $R_{\text{exit}} = 50$. The following providers are registered:

- `P1`: stake just above `MINIMUM_PROVIDER_STAKE`, so that one slash drops it below the minimum.
- `P2`: a large stake, so that slashing alone will not drop it below the minimum during the test.
- `P3`: a reputation score low enough that one decrease brings it to 0, and a stake of 100 tokens.
- `P4`, `P5`, and `P6`: 100 tokens of stake each and reputation scores of 80, 50, and 25.

Input:

1. Run `REPEAT_OFFENSE_WINDOW` requests for the task. In every request, most providers submit predictions close to 50. `P1` submits nothing in the first request. `P2` submits nothing in `REPEAT_OFFENSE_COUNT` of the requests. `P3` submits a prediction far from the others (5000) in the first request.
2. While one of the requests is active, `P4` tries to unregister.
3. After the requests finish, `P4`, `P5`, and `P6` voluntarily unregister.

Output:

1. `P1` is no longer in the task's list of registered providers after the first request, because its stake fell below `MINIMUM_PROVIDER_STAKE`. `P2` is no longer registered after its `REPEAT_OFFENSE_COUNT`-th slash, and its remaining stake is burned. `P3`'s reputation is 0 after the first request, `P3` is no longer registered, and its remaining stake is burned.
2. `P4`'s attempt to unregister is refused with an error message, and `P4` stays registered.
3. `P4` ($R = 80$) and `P5` ($R = 50 = R_{\text{exit}}$) each get all 100 tokens back. `P6` ($R = 25$) gets $\frac{25}{50} \times 100 = 50$ tokens back, and the other 50 are burned.

Test Case Derivation: FR56, FR57, and FR58 give the three cases where a provider is removed automatically: stake below the minimum, too many slashes, and a reputation of 0. FR57 and FR58 also require the remaining stake to be burned. FR14 does not allow a provider to unregister while it has an active request. FR59 sets how much stake is returned on voluntary unregistration, and `P5` sits exactly on the threshold.

How test will be performed: After each request, record each provider's registration status, stake, and slash count. Record the total burned token supply and each provider's token balance before and after unregistration, and assert the expected changes.

5.1.8 Transparency and Data Handling

These tests verify that users can see and verify the transactions they authorize (FR60–FR63) and that a request's input data is protected for its whole life (FR64–FR66). They follow a requester who pays for a request and checks it, and a request's input data from submission to deletion.

1. ST-TRANS-1: Requester pays for a request and verifies it

   Control: Automatic

   Initial State: An authenticated requester is signed in with a test wallet signer that holds 50 tokens. Task `Rep-Test-Task` and the providers from ST-REP-1 are registered.

   Input:

   1. Submit an inference request with a payment of 137 tokens.
   2. Add funds so the wallet holds 1000 tokens, then submit the request again and approve the transaction.
   3. Let the request run to completion, then open the transaction history.

   Output:

   1. On the first attempt, the system shows an insufficient-funds error and does not ask the wallet for a signature.
   2. On the second attempt, before the wallet asks for a signature, the application shows the action ("Submit inference request") and the amount (137 tokens).
   3. The transaction status is shown as pending and then confirmed.
   4. The requester's token balance is exactly 137 tokens lower than before the payment.
   5. The transaction history shows an identifier for the payment. Looking it up on the public blockchain network returns a transaction that:
      - is signed by the requester's wallet;
      - transfers exactly 137 tokens from the requester's token account to the request's escrow address.
   6. After the request is completed, the blockchain shows:
      - the 137 tokens arriving at the request's escrow address, and then being transferred from the escrow address to the providers' token accounts, with transfers that add up to no more than 137 tokens;
      - a stake account for each provider that holds the stake shown on that provider's dashboard;
      - the final aggregated result stored for the request, equal to the result shown to the requester;

- each provider's reputation after the request, equal to the reputation shown in the application.

Test Case Derivation: FR.SA.2 requires a balance check before a signature is requested. FR4 requires the requester to sign the payment, FR60 requires the action and amount to be shown before signing, and FR61 requires the status to be shown until it is final. FR.SA.8 requires the amount charged to equal the amount approved; an unusual amount (137) is used so that a hard-coded or rounded value would be caught. FR62 requires an identifier that can be looked up on the blockchain, and FR63 lists the records that must be on the blockchain. The escrow is checked by its transaction history instead of its balance, because the tokens may already have been paid out by the time it is checked.

How test will be performed: Run the flow in an automated browser test. Pause the test wallet signer when it receives a signing request and check the confirmation screen. Record each status shown. Compare balances before and after. Query the public blockchain network with the transaction identifier, and read the escrow address's transaction history and the request, stake, and reputation accounts directly, without going through the application's backend.

2. ST-DATA-1: Input data from submission to deletion

Control: Automatic

Initial State: Task `Data-Test-Task` has registered providers `P1`, `P2`, and `P3`. Another authenticated user `U1` is not involved.

Input:

1. Requester `R1` submits request `REQ-A` whose input contains the marker value `271 828.1828`. `P1` and `P2` accept it; `P3` does not.
2. During the commit and reveal phases:
   - `R1` opens `REQ-A` in the web client.
   - `P1`, `P2`, and `P3` each call the provider SDK's method for fetching a request's input and try to decrypt the result with their own private key.
   - `U1` and an unauthenticated client call the backend's input-data endpoint for `REQ-A` directly.
3. `REQ-A` runs to completion.
4. `R1` submits request `REQ-B`, which is cancelled because too few providers accept it.

Output:

1. `R1` sees `REQ-A`'s input in the web client. `P1` and `P2` receive the encrypted input and can decrypt it to the original values. `P3`'s SDK call is refused, or returns data

`P3` cannot decrypt. `U1` and the unauthenticated client are refused and receive no part of the input.

2. No transaction or account linked to either request contains the marker value in readable form.
3. After each request reaches its final state, its input is no longer in any storage the system controls, and the web client, the SDK, and the backend endpoint all return "not found" for it, including for `R1`.

Test Case Derivation: FR64 limits the input to the requester and the providers that accepted; `P3` checks that being registered is not enough. FR65 forbids readable input on the blockchain, and the marker value makes it possible to search raw on-chain data for it. FR66 requires deletion after both completion and cancellation. The test also exercises FR26 (input given to accepting providers) and FR24 (cancellation when too few providers accept).

How test will be performed: Run each access attempt and compare what is returned with the original input. Search the raw data of every linked transaction and account for the marker in its common encodings (text, 32-bit and 64-bit floating point, and the program's fixed-point format). After each request ends, query the database, file storage, and caches directly and through the application.

5.1.9 Safety

This test verifies the transaction safety requirements that are not already covered by the tests above (FR.SA.1 and FR.SA.3–FR.SA.7). FR.SA.2 and FR.SA.8 are covered by ST-TRANS-1, and FR.SA.9 and FR.SA.10 by ST-REP-1. The test injects faults using a test build of the backend and a proxy between the system and the public blockchain network RPC.

1. ST-SAFE-1: Requester payments under transaction faults

   Control: Automatic, with the retry step also checked manually in the web client

   Initial State: A requester is signed in with 1000 tokens. The RPC proxy and the backend test build can each be set to inject one fault.

   Input:

   1. The requester submits an inference request payment through a proxy that forwards the transaction but drops the response.
   2. The requester submits an inference request payment through a proxy that holds the transaction until its blockhash has expired.
   3. The requester submits an inference request payment, and the requester's tokens are moved out of the wallet after the balance check but before the transaction runs.

4. The requester submits an inference request payment and signs it with a different wallet address than the one connected to their account.
5. The requester submits an inference request payment while the backend test build changes the transaction's amount to 150 tokens after 100 tokens has been shown to the user.
6. After the failure in step 2, the requester presses retry.

Output:

1. Exactly one payment transaction from the requester appears on the blockchain, the requester's balance goes down by the payment amount once, and the transaction log shows each status the payment went through.
2. The payment is shown as failed only after its blockhash has expired, and no payment from this attempt appears on the blockchain.
3. The payment transaction fails on the blockchain, no request is created, and the requester's escrow and request accounts are unchanged.
4. No transaction is submitted to the blockchain, and the user sees an error message saying the transaction was signed by the wrong wallet.
5. The web client does not send the transaction to the wallet for signing, and the user sees an error message saying the transaction does not match the amount shown.
6. The user sees an error message with an option to retry. After pressing retry, the request is created and exactly one payment for it appears on the blockchain.

Test Case Derivation: Each fault matches one requirement:

1. FR.SA.1 forbids resubmitting before confirming the first transaction was not processed, and requires status logs.
2. FR.SA.5 requires the system to wait for the validity period to end and check the status before logging a failure.
3. FR.SA.3 requires the on-chain program to check funds when the transaction runs.
4. FR.SA.4 forbids submitting a transaction that is not signed by the authorized wallet.
5. FR.SA.7 requires the transaction to be simulated and checked against the displayed amount before it is signed. The error is introduced by the backend, and the check is done by the web client, which compares what the user is about to sign with what is displayed.
6. FR.SA.6 requires the error to be shown and a retry to be allowed.

How test will be performed: Run each fault in turn. Use the proxy to count submitted transactions and the test wallet signer to count signing requests. For step 2, compare the time the payment is shown as failed with the block height at which its blockhash expired, and check that the backend queried the transaction's status before marking it as failed. For step 4, configure the test wallet signer to sign with a second test wallet. Count payments from the requester's wallet on the public blockchain after the whole test and compare the final balance with the number of successful requests. A tester also runs step 6 by hand in the web client and records the error message shown.

5.2 Tests for Nonfunctional Requirements

This section describes the tests for the non-functional requirements in the SRS (NFR1–NFR18 and NFR.SA.1–NFR.SA.4). As with the functional tests, each test is a larger activity that checks several related requirements at once: a large simulation of many requests, a usability study, a security and privacy audit, and a set of release checks. Some tests produce measurements, tables, or graphs rather than a single pass or fail result. Values that have not been decided yet are written as symbolic constants and are listed in the Symbolic Parameters appendix.

5.2.1 Accuracy, Reliability, and Scalability

1. NFR-SIM-1: Large-scale request simulation

   Control: Automatic

   Initial State: The system runs on a local validator for the public blockchain network with a simulation harness that controls mock providers and can inject faults. A separate reference implementation of the Appendix A aggregation algorithm is written in TypeScript by a team member who did not write the system's version.

   Input:

   1. Run `NUM_SIMULATED_REQUESTS` requests with random predictions and reputation scores, including values at the edges of their declared ranges (reputation 0 and 100, the largest and smallest allowed predictions and token amounts) and the worked examples from the SRS.
   2. Mix in failure scenarios: too few providers accept, too few valid predictions, providers that do not commit or reveal, mismatched commitments, invalid revealed predictions, dropped or delayed RPC responses, and a backend restart during the commit and reveal phases.
   3. Run the requests at 1, 5, 10, 20, and `MAX_CONCURRENT_REQUESTS` at the same time. Mock providers reply immediately, so only the system's own processing time is measured.

4. During each commit phase, try to read any individual prediction from on-chain and backend data as another provider, as the requester, and from the raw on-chain data; include the backend database and logs.
5. Generate a salt for each mock provider. Inspect the SDK-generated salt length and persistence before and after a simulated restart, then use the same salt for reveal.

Output:

1. Every aggregated result equals the reference implementation's result. The test reports the number of mismatches (expected: 0).
2. No calculation overflows. Any input that would overflow causes the transaction to fail with no change to on-chain state.
3. Every request reaches a final state before its final deadline, and its escrow is empty afterwards.
4. No individual prediction can be read from on-chain or backend data before the commit phase ends.
5. Each salt contains at least 256 bits, is generated by a cryptographically secure random generator, and remains available in the SDK after restart until reveal completes.
5. A table and graph of the median and 95th-percentile processing time at each concurrency level. The test passes if the median at `MAX_CONCURRENT_REQUESTS` is no more than `PERFORMANCE_DEGRADATION_TOLERANCE` higher than for a single request.

Test Case Derivation: NFR1 requires simulation tests to show that the results match the Appendix A algorithm. NFR2 forbids overflow and requires an overflowing transaction to leave no change. NFR14 requires every request to reach a final state and settle its escrow before the final deadline. NFR11 requires predictions to stay hidden until the commit phase ends. NFR12 requires a cryptographically secure 256-bit salt to persist until reveal. NFR16 requires concurrent requests without a drop in performance.

How test will be performed: The harness runs the requests and then reads each request's status, result, and escrow from the blockchain. It compares each result with the reference implementation and logs the inputs of any mismatch so they can be added as a unit test. It checks that the tokens escrowed at the start equal the tokens paid out, returned, slashed, and burned. Each concurrency level is run 3 times, and the timings come from the system's logs.

5.2.2 Usability

1. NFR-USE-1: Usability study with new users

Control: Manual, with timing checks automated

Initial State: The web application is deployed on a test environment connected to the public blockchain network. 5 to 8 participants who use a web browser but have never used a blockchain wallet or built a machine learning model are recruited.

Input: Each participant gets a short description of DAIIM, but no instructions on how to use it, and completes these tasks while thinking out loud:

1. Sign in by creating an embedded wallet with an email address.
2. Find the documentation and the list of available tasks.
3. Submit an inference request for an existing task.
4. Find the result of the request and the payment for it.
5. Sign out.

During task 3, the test build forces one error (a failed transaction). The participant then fills out the survey in the Usability Survey Questions appendix.

Output:

1. At least `USABILITY_COMPLETION_RATE` of participants complete every task without help, including creating a wallet with no Web3 knowledge.
2. The median time to finish tasks 1 to 3 is 5 minutes or less.
3. Every feature can be reached within 2 clicks of the home page.
4. Every error shown has a name, a description, and a suggested fix, and appears within 2 seconds of the failure being detected.
5. A progress indicator is shown during every operation longer than `LONG_OPERATION_THRESHOLD`.
6. A plain-language confirmation is shown after every transaction.
7. The survey results are summarized in a table and bar chart.

Test Case Derivation: NFR3 requires a new user to learn the system within 5 minutes, and NFR8 requires them to create an embedded wallet without Web3 knowledge. NFR4 requires every feature within 2 clicks. NFR5 requires errors to have a name, description, and fix, and NFR.SA.1 requires them within 2 seconds. NFR6 requires progress to be shown for long operations, and NFR7 requires plain-language confirmations.

How test will be performed: A team member observes each session, times each task, records clicks, and only helps if the participant is stuck for more than 3 minutes (which counts as not completing the task). A team member also counts the clicks to each feature from the home page. Error timings and progress indicators are checked by an automated browser test that repeats the same tasks with forced errors and delays. One study is held after the Rev 0 demo and a second after changes are made in response to it.

5.2.3 Security, Privacy, and Safety

1. NFR-SEC-1: Security and privacy audit

Control: Manual, with automated scans

Initial State: The system is deployed in its test environment. A request is in progress, so its input data is stored. Test providers have consented to showing different sets of profile information.

Input:

1. Run a full request with an extension wallet and with an embedded wallet, capturing network traffic and backend logs.
2. Read the raw contents of the database and file storage.
3. Scan every endpoint's TLS setup, connect over plain HTTP, and point the backend at RPC and storage endpoints that use a self-signed certificate.
4. Visit every public page without signing in.

Output:

1. Input data is encrypted in all captured traffic and cannot be read in raw storage, using algorithms and key lengths on a recognized recommended list (for example, NIST SP 800-175B).
2. All endpoints use TLS 1.2 or later with valid certificates. Plain HTTP is refused or redirected, and the backend refuses endpoints with invalid certificates.
3. No private key or recovery phrase appears in traffic, logs, or storage, and no email address or phone number appears on the blockchain.
4. Public pages show only the information each provider agreed to share.
5. A completed audit checklist based on the PIPEDA safeguards principle, with any gaps recorded as issues.

Test Case Derivation: NFR9 and NFR10 require sensitive data to be encrypted in transit and at rest to an industry standard, verified through a security audit. NFR.SA.3 requires TLS with certificate checks. NFR.SA.2 requires all signing to happen in the user's wallet with no private information on the blockchain. NFR13 limits public pages to information providers consented to.

How test will be performed: A team member who did not write the storage code leads the audit. Search the captured traffic, logs, storage, and on-chain data for marker values in the input and for the test wallets' keys and the test users' contact details. Run `testssl.sh` against each endpoint. Compare each public page with the providers' consent settings. The team reviews the checklist in a meeting.

5.2.4 Maintainability, Portability, and Installability

1. NFR-RELEASE-1: Release checks

   Control: Automatic, with a manual run on Safari

   Initial State: A release candidate is on a pull request to `main`, and the GitHub Actions workflow is set up.

   Input:

   1. CI runs `ESLint`, `Prettier--check`, `cargoclippy` (with warnings treated as errors), `cargofmt--check`, `npmaudit`, and `cargoaudit`.
   2. The main user flow (sign in with an embedded wallet, browse tasks, submit a request, view the result and transaction history, sign out) is run on the current versions of Chrome, Firefox, Edge, and Safari, each in a new browser profile with no extensions installed.

   Output:

   1. The linters and formatters report no errors, and the pull request cannot be merged while any check fails.
   2. No dependency vulnerability of high or critical severity is reported unless the team has reviewed it and recorded it in an issue.
   3. The flow completes in every browser with no console errors and without asking the user to install any software or extension. The result is a table of each browser and step.

   Test Case Derivation: NFR15 requires the coding standards in the Development Plan, enforced by linting; the Development Plan names these tools. NFR.SA.4 requires dependencies to be scanned for known vulnerabilities. NFR17 requires the system to work on current modern browsers, and NFR18 requires embedded-wallet users to need nothing beyond a browser.

   How test will be performed: The linters and scans run in CI on every pull request and once a week, and branch protection on `main` requires them to pass. The browser flow runs as an automated test on Chrome, Firefox, and Edge in CI. A team member runs it by hand on Safari, since Safari cannot run in CI. Reviewers check the standards that linters cannot (such as `rustdoc` comments) during pull request review.

5.3 Traceability Between Test Cases and Requirements

The following table summarizes the traceability from each system test to the requirements it supports. The mappings are derived from the Test Case Derivation for each test and correspond to the functional and non-functional requirements in the SRS.

Table 4: Traceability Between Test Cases and Requirements

System test	Supported requirements
ST-AUTH-EXT-1	FR1
ST-AUTH-EXT-2	FR1
ST-AUTH-EMB-1	FR1, FR2
ST-AUTH-EMB-2	FR1
ST-TASK-CREATE-1	FR5, FR6, FR7, FR9, FR10, FR8, FR4, FR60, NFR7
ST-TASK-BROWSE-1	FR3, FR11, FR5, FR18
ST-PROV-1	FR12, FR13, FR4, FR60, FR61, FR62, NFR7
ST-PROV-2	FR12, FR13
ST-PROV-3	FR14, FR59
ST-PROV-4	FR14
ST-PROV-5	FR15
ST-REQ-1	FR18
ST-REQ-2	FR19, FR18
ST-REQ-3	FR20, FR4
ST-REQ-4	FR16, FR17, FR21
ST-ACC-1	FR22, FR4, FR33, FR10
ST-ACC-2	FR22, FR23, FR7
ST-ACC-3	FR24, FR34
ST-ACC-4	FR33, FR34
ST-PRED-1	FR28, FR31, NFR12
ST-PRED-2	FR27, FR29, FR30, FR31
ST-PRED-3	FR32, FR35, FR34
ST-AGG-1	FR36, FR30, FR37, FR38
ST-AGG-2	FR39, FR37, FR38, FR47, FR51, FR24
ST-REP-1	FR40, FR42, FR43, FR44, FR46, FR51, FR.SA.9, FR.SA.10
ST-REP-2	FR47, FR48, FR49, FR50, FR51
ST-PEN-1	FR52, FR53, FR54, FR55, FR45, FR41
ST-PEN-2	FR14, FR56, FR57, FR58, FR59
ST-TRANS-1	FR4, FR60, FR61, FR62, FR63, FR.SA.2, FR.SA.8
ST-DATA-1	FR26, FR64, FR65, FR66
ST-SAFE-1	FR.SA.1, FR.SA.3, FR.SA.7, FR.SA.5, FR.SA.4, FR.SA.6
NFR-SIM-1	NFR1, NFR2, NFR11, NFR12, NFR14, NFR16
NFR-USE-1	NFR3, NFR4, NFR5, NFR6, NFR7, NFR8, NFR.SA.1

System test	Supported requirements
NFR-SEC-1	NFR9, NFR10, NFR13, NFR.SA.2, NFR.SA.3
NFR-RELEASE-1	NFR15, NFR17, NFR18, NFR.SA.4

6 Unit Test Description

The unit test plan will be added upon completion of the MIS.

7 Appendix

7.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

The constants used by the system itself (`MINIMUM_PROVIDER_STAKE`, `MAX_REPUTATION_PENALTY`, `SLASH_FRACTION`, `REPEAT_OFFENSE_COUNT`, `REPEAT_OFFENSE_WINDOW`, `USABILITY_COMPLETION_RATE`, `COMMIT_PERIOD`, and `REVEAL_PERIOD`) are defined in Appendix C of the SRS. A test deployment uses one value for each protocol constant across all requests in that deployment; isolated test runs may set different values. The constants below are only used by the tests in this document.

- `TASK_RESERVATION_AMOUNT`: the amount of provider stake that is reserved when a provider accepts a request and becomes an accepted provider. Proposed value: 10.
- `REQUEST_PAYMENT`: the amount a requester pays to submit a request and receive an aggregated prediction, including transaction fees and provider payment. Proposed value: 40.
- `MINIMUM_REQUEST_PAYMENT`: the amount a provider is guaranteed to be paid for a given request, if a valid prediction is submitted. Proposed value: 10.
- `LONG_OPERATION_THRESHOLD`: the length of time after which an operation counts as "long" and must show progress (NFR-USE-1). Proposed value: 1 second.
- `NUM_SIMULATED_REQUESTS`: the number of requests run in the reliability simulation (NFR-SIM-1). Proposed value: 500, spread across the listed scenarios.
- `MAX_CONCURRENT_REQUESTS`: the largest number of concurrent requests in the scalability test (NFR-SIM-1). Proposed value: 50.
- `PERFORMANCE_DEGRADATION_TOLERANCE`: the largest allowed increase in median processing time at `MAX_CONCURRENT_REQUESTS` compared with a single request (NFR-SIM-1). Proposed value: 25%.

7.2 Usability Survey Questions

Participants answer this survey at the end of each usability session described in test NFR-USE-1. Questions 1 to 8 use a scale from 1 (strongly disagree) to 5 (strongly agree).

1. It was easy to sign in and create a wallet.
2. I understood what a wallet was used for in DAIIM after signing in.
3. I could find the features I needed without searching for a long time.

4. I understood what I was paying for before I approved each payment.
5. When something went wrong, the error message told me what happened and what to do next.
6. While waiting for something to finish, I could tell that the system was still working.
7. After each payment, I understood what had happened and whether my balance had changed.
8. I would be comfortable using DAIIM again without help.
9. Which task was the hardest to complete, and why?
10. Were there any words or messages you did not understand? Which ones?
11. Was there anything you expected to be able to do but could not?
12. Do you have any other comments about the application?

8 Appendix — Generative AI Use Disclosure

8.1 AI Tools Used

An AI assistant using the Copilot SDK in Visual Studio Code was used to assist with revisions to this document. The specific model used is GPT-6 Luna.

8.2 How and Where Used

The AI assistant was used to discuss, draft, and revise prose and requirements for the verification and validation work. In the VnV Plan, it helped structure system tests by functional area and draft and refine test cases for wallet authentication, embedded-wallet creation, returning-user authentication, and inference-task creation. Revisions focused on making inputs, expected outputs, test procedures, and requirement derivations clearer and more specific.

The assistant also helped revise related SRS content: clarifying support for embedded and browser-extension wallets, adding requirements for embedded-wallet creation and persistent wallet-to-profile association, and adding a conservative User Profile entry to the major data entities and data dictionary. The team directed the scope and reviewed or adjusted the proposed content.

8.3 Verification of AI-Generated Content

AI-assisted content in this document was checked against the SRS to confirm that each test case addresses stated requirements and does not introduce unapproved system behaviour or design decisions. Requirement references and the expected outcomes derived from them were checked against the corresponding SRS requirements and use cases. Test cases were reviewed for clear inputs, expected results, and procedures, and the LaTeX structure and cross-references were inspected after editing. Test fixtures, schema examples, and blockchain verification details are proposed examples and must be reconciled with the implemented system before execution. The team remains responsible for the accuracy, completeness, and consistency of this document, regardless of whether material was first drafted by a team member or with the help of an AI tool.

9 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

   Aidan: Splitting the system tests by SRS subsection worked well. Because our functional requirements were already grouped by the phase of an inference request, each of us could take a set of subsections and write the tests for them without waiting on anyone else. Writing tests also made the SRS easier to check. For the reputation and reward tests, I worked out small sets of predictions by hand, so that each provider's distance from the result fell into a known case (increase, no change, or decrease). Doing this showed that the reputation rules in the SRS are precise enough to test, and it gave us expected values we can reuse when we write the automated tests later.

   Robert: What went well for me while writing this deliverable was taking the requirements for authentication using both embedded and traditional wallets and creating system tests for these. The suggestion to consider using embedded wallets for DAIIM came from our supervisor Dr. Hassan, as they help to abstract away the complexities of setting up a cryptocurrency wallet for users who are not familiar with the technology. Because the requirement for authentication using embedded wallets only recently added to the SRS, writing the system tests forced me to better understand how embedded wallets work in practice, and the important aspects to consider for validation.

   Keira: Something that went well when writing this document was the conversation it sparked with our supervisor, Dr. Hassan. Section 3 prompted us to think more concretely about development of the system and the roles each team member would play. As a result of this, we were inspired to discuss with Dr. Hassan what kind of guidance and expertise he would reasonably be able to offer to us for the remainder of the project. We planned structured meetings in which we will demostrate our PoC and Rev 0, and Dr. Hassan will offer advise and critiques from the perspective of the stakeholder, as well as a subject matter expert.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Aidan: The main pain point was that some parts of the SRS were not yet specific enough to test. For example, several constants such as `SLASH_FRACTION` and `MAX_REPUTATION_PENALTY` are still "to be determined", and the reward requirement does not say whether rewards are split using reputation scores from before or after the request's reputation update. To handle this, I wrote the tests in terms of the symbolic constants so they will not need to change once the values are chosen, and I wrote down the assumption made in the reward test so the team can confirm it and update the SRS.

   Another pain point was not knowing how much detail the system tests needed, or whether every requirement needed its own test. My first draft had one small test for almost every requirement, which made my sections very long and repeated the same setup many times. After comparing it with an example VnV plan from another capstone group, I realized that system tests should follow larger, realistic scenarios, like a requester paying for a request and checking it on the blockchain, or a group of providers misbehaving during one request. I rewrote my sections so that each test covers about five to seven related requirements, and left the small edge cases, such as a distance exactly equal to the average, for the unit tests. To make sure nothing was lost in the rewrite, I checked that every requirement in my sections was still named in at least one test. We also added the question of how detailed the system tests should be to our list of questions for our TA.

   Robert: The main pain point I noticed when writing this deliverable was creating and maintaing a set of test fixtures used throughout the system tests. The fixtures are used to provide a consistent set wallets, user profiles, inference task etc. for the tests, but as more and more tests were added to the document, it became difficult to keep track of which fixtures were being used in which tests, and what each one represented. We decided, to resolve this, to avoid reusing fixtures across tests, and instead create a new fixture, with a new ID, each time.

   Keira: The main pain point I had was keeping the system tests independent of the blockchain platform. My first drafts named Solana Devnet directly and used fixed values such as a stake of 100 SOL. After review feedback, I had to remove these references. I resolved this by replacing fixed amounts with symbolic constants, such as `REQUEST_PAYMENT` and `MINIMUM_REQUEST_PAYMENT`, and defining them once in the appendix. Now a test describes what must be true, not how one platform does it, and changing a value only means updating the appendix. In hindsight, deciding on this convention before writing the tests would have saved most of the rework.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project?

   - **Testing Solana programs (Robert):** Writing automated tests for our on-chain programs with the Anchor test framework and a local Solana validator, including

setting up accounts and checking account state before and after a transaction.

- **Simulation and fault injection (Aidan):** Building a simulation harness with mock providers that can run many requests and inject failures, such as dropped RPC responses or providers that never submit. This is needed for the reliability, scalability, and safety tests.
- **Usability testing (Keira):** Planning and running usability sessions, observing participants without leading them, and turning survey answers and observations into changes to the interface.
- **Security testing (Nathan):** Checking encryption in transit and at rest, scanning TLS configurations, searching network traffic and on-chain data for leaked information, and running dependency scans.
- **Frontend and end-to-end testing (all):** Writing Vitest tests for the React frontend and automated browser tests that sign transactions with a test wallet.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   - **Testing Solana programs:** (1) Work through the testing sections of the Anchor documentation and its example projects, or (2) read the tests of existing open-source Solana programs. Robert will start with the Anchor documentation, since he already has some Solana experience and the documentation matches the version of Anchor we use.
   - **Simulation and fault injection:** (1) Build a small prototype harness during the proof of concept and grow it, or (2) study how existing tools inject network faults, such as proxy-based tools. Aidan will build a prototype during the proof of concept, because the harness depends on our own request flow, and starting small will show early which faults are hard to set up.
   - **Usability testing:** (1) Apply the methods from SFWRENG 4HC3, or (2) run a small pilot session with a classmate before the real sessions. Keira will use the 4HC3 material, since she is taking the course now, and the team will run a pilot session to test the tasks and survey.
   - **Security testing:** (1) Follow guides such as the OWASP Web Security Testing Guide, or (2) learn from the cryptography and network security courses some members have taken. Nathan will follow the OWASP guide because it gives step-by-step tests, and he will ask Robert and Keira for help with the cryptography and network questions their courses cover.
   - **Frontend and end-to-end testing:** (1) Read the Vitest and browser-testing tool documentation, or (2) pair program the first few tests and then review each

other's tests in pull requests. The team will pair program the first tests, so everyone learns the same patterns, and then rely on pull request reviews to keep the tests consistent.